➢ Define Problem Statement and perform Exploratory Data Analysis

➔ Definition of problem (as per given problem statement with additional views)

Scaler is a transformative online education platform designed to help software professionals upskill and accelerate their careers. It offers intensive live classes taught by tech leaders and industry experts, providing hands-on experience with a modern curriculum tailored to meet current industry demands. The platform specializes in technical education, particularly in software engineering and data science, with a focus on problem-solving, system design, and real-world projects.

Scaler's mission is to bridge the gap between academia and the industry by preparing learners for high-demand roles in top companies. By combining mentorship, structured programs, and cutting-edge technologies, Scaler has become a preferred platform for professionals seeking career growth.

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [3]:
```
df = pd.read_csv("C:\SCALER\Case_Studies\9_clusturing\scaler_clustering.csv")
df.sample(5)
```

Out[3]:

	Unnamed: 0	company_hash	email_hash	orgyear	ctc	job_position	ctc
144940	145488	vagmt	0da4cf18eac7fc00643c4f696629aa230ecc6bd675fb89...	2014.0	4000000	Backend Engineer	
59897	59976	xzegojo	b1f8f9184fe1d473b4992c5bb76e174b49ac5c56930177...	2018.0	360000	NaN	
204395	205473	opj	f149b53938f040620b71b4beb7ded5a221c39085e09ee9...	2014.0	1400000	NaN	
171720	172505	sgk ucn rna	d26b77cd4500205f912c790444957fa23932831044ab0f...	2013.0	1390000	Android Engineer	
49461	49516	gqvzt ntwy	75875ee7e65d3aa68122d387f1363432c0a9c579feb694...	2016.0	1700000	Support Engineer	

Data Dictionary:

Email_hash - Anonymised Personal Identifiable Information (PII)

Company_hash - This represents an anonymized identifier for the company, which is the current employer of the learner.

orgyear - Employment start date

CTC - Current CTC

Job_position - Job profile in the company

CTC_updated_year - Year in which CTC got updated (Yearly increments, Promotions)

==========

➔ Observations on shape of data, data types of all the attributes, conversion of categorical attributes to 'category' (If required) , missing value detection, statistical summary.

In [4]:
```
1 df.shape
```

Out[4]:
```
(205843, 7)
```

In [5]:
```
1 df.dtypes
```

Out[5]:
```
Unnamed: 0           int64
company_hash        object
email_hash          object
orgyear            float64
ctc                  int64
job_position        object
ctc_updated_year   float64
dtype: object
```

In [6]:
```
1 df.duplicated().sum()
```

Out[6]:
```
0
```

In [7]:
```
1 df.isnull().sum().sort_values(ascending=False)
```

Out[7]:
```
job_position        52564
orgyear                86
company_hash           44
Unnamed: 0              0
email_hash              0
ctc                     0
ctc_updated_year        0
dtype: int64
```

In [8]:
```
1 df.describe()
```

Out[8]:

	Unnamed: 0	orgyear	ctc	ctc_updated_year
count	205843.000000	205757.000000	2.058430e+05	205843.000000
mean	103273.941786	2014.882750	2.271685e+06	2019.628231
std	59741.306484	63.571115	1.180091e+07	1.325104
min	0.000000	0.000000	2.000000e+00	2015.000000
25%	51518.500000	2013.000000	5.300000e+05	2019.000000
50%	103151.000000	2016.000000	9.500000e+05	2020.000000
75%	154992.500000	2018.000000	1.700000e+06	2021.000000
max	206922.000000	20165.000000	1.000150e+09	2021.000000

In [9]:
```
1 df.describe(include=['object'])
```

Out[9]:

	company_hash	email_hash	job_position
count	205799	205843	153279
unique	37299	153443	1016
top	nvnv wgzohrnvzwj otqcxwto	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7...	Backend Engineer
freq	8337	10	43554

==========

➢ Filling missing values

In [10]:
```
df.sample(7)
```

Out[10]:

	Unnamed: 0	company_hash	email_hash	orgyear	ctc	job_position	ctc_
113039	113326	vuurxta bvntqxvro xzaxv ucn rna	824021f77554feb50595f446a2fc045d065d67a9bce81d...	2016.0	1160000	NaN	
184647	185552	btqwtatomtzk qtotvqwy vza atctrgubtzn xzaxv	2f1dbf982318989237690d68f2607ce90478d662c46813...	2019.0	1030000	Devops Engineer	
18969	18981	xzegojo	44369ece2256fca17f38ca1c2d17f533e155771e76a24c...	2022.0	500000	Product Designer	
38443	38487	vbvkgz	cf4f76771f4112a08b4ec6920db1d2eb740a1518470248...	2018.0	2750000	Engineering Intern	
56957	57029	ntvwygg	f87ef758894f9b25780885a1520d8135e610d607ae9514...	2019.0	1050000	NaN	
201554	202606	oou fgqrafxat	c1c82a84736abadd85fbbf567e31380ec3baf85c67c60e...	2013.0	650000	QA Engineer	
96434	96624	iexd xzegwgbb	ab8bbd7332d67fd2eb9838eaa5eaa04158152c702f4692...	2020.0	200000	NaN	

In [11]:
```
import re

# Apply the regex cleaning to 'email_hash' column
df['email_hash_cleaned'] = df['email_hash'].apply(lambda x: re.sub('[^A-Za-z0-9 ]+', '', str(x)

df = df.drop(columns=['email_hash'])
df.head()
```

Out[11]:

	Unnamed: 0	company_hash	orgyear	ctc	job_position	ctc_updated_year	email_hash
0	0	atrgxnnt xzaxv	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149
1	1	qtrxvzwt xzegwgbb rxbxnta	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5
2	2	ojzwnvwnxw vx	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd9817
3	3	ngpgutaxv	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d6
4	4	qxen sqghu	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f7

In [12]:
```
# Apply the regex cleaning to 'company_hash' column
df['company_hash_cleaned'] = df['company_hash'].apply(lambda x: re.sub('[^A-Za-z0-9 ]+', '', st

df = df.drop(columns=['Unnamed: 0','company_hash'])
df.head()
```

Out[12]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeg rx
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpg
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen

In [13]:
```
df.isnull().sum().sort_values(ascending=False)
```

Out[13]:
```
job_position            52564
orgyear                    86
ctc                         0
ctc_updated_year            0
email_hash_cleaned          0
company_hash_cleaned        0
dtype: int64
```

In [14]:
```
duplicates_count = df.duplicated().sum()
print(f"Number of duplicates: {duplicates_count}")

df = df.drop_duplicates()

updated_duplicates_count = df.duplicated().sum()
print(f"Number of duplicates after removal: {updated_duplicates_count}")
```

```
Number of duplicates: 34
Number of duplicates after removal: 0
```

In [15]:
```
df.sample(10)
```

Out[15]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_ha
13654	2016.0	600000	Program Manager	2020.0	54c1303e9dedac943ee69a646410804d77ba46cdb24788...	qtaxzsngz x
170443	2018.0	950000	Backend Engineer	2020.0	f1738ea810901155337d4d1388626d64ee060f46a5a440...	vowt atctrgubtzn
106791	2018.0	3000000	Backend Engineer	2019.0	69420bef0d4b709cee2d50288e38971e93908685af6c9a...	
52092	2017.0	70000	FullStack Engineer	2017.0	0670aad4e8690c8331e33a4ccbc51339d7f17a1dae7f78...	
130730	2015.0	2270000	FullStack Engineer	2018.0	d8ec283b254429ae833a37923ac19c341e19930a0939b6...	
33322	2014.0	1500000	NaN	2021.0	8c99738310d934cf75208f4fee02fbcec7c4939fe83a8f...	wjn
44461	2016.0	900000	NaN	2020.0	0920ba54e3516458e42d25816590e35be05f939b699914...	
77345	2014.0	1000000	NaN	2021.0	9e67a69a43fae5b6d42a22172cfc79b6b4b3ab176e21c9...	bjonxr
4883	2013.0	6000000	NaN	2021.0	f033d0c699954c9cbfff1ee9f1d819ace32323f9fbfd8c...	
158318	2019.0	3200000	Data Scientist	2018.0	7b48a8e29d116bb1ed6c2728da80c20b426c5041570286...	

In [19]:
```
filtered_df = df[df['orgyear'].astype(str).str.len() != 4]

filtered_df
```

Out[19]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_ha
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	at
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzv
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojz
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	
...	...	...	...	...	...	
205838	2008.0	220000	NaN	2019.0	70027b728c8ee901fe979533ed94ffda97be08fc23f33b...	
205839	2017.0	500000	NaN	2020.0	7f7292ffad724ebbe9ca860f515245368d714c84705b42...	
205840	2021.0	700000	NaN	2021.0	cb25cc7304e9a24facda7f5567c7922ffc48e3d5d6018c...	
205841	2019.0	5100000	NaN	2019.0	fb46a1a2752f5f652ce634f6178d0578ef6995ee59f6c8...	zgn
205842	2014.0	1240000	NaN	2016.0	0bcfc1d05f2e8dc4147743a1313aa70a119b41b30d4a1f...	bg

205804 rows × 6 columns

In [20]:
```
df.shape
```

Out[20]: (205809, 6)

In [21]:
```
l=list
l= list(df['orgyear'].nsmallest(50))

# Display the result
print(l)
```

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 2.
0, 2.0, 2.0, 3.0, 3.0, 3.0, 3.0, 3.0, 3.0, 4.0, 5.0, 5.0, 6.0, 6.0, 38.0, 83.0, 91.0, 91.0, 91.0,
200.0, 201.0, 206.0, 208.0, 209.0, 1900.0, 1970.0, 1970.0, 1971.0, 1972.0, 1973.0, 1976.0]
```

In [22]:
```
ll=list
ll= list(df['orgyear'].nlargest(50))

# Display the result
print(ll)
```

```
[20165.0, 20165.0, 2204.0, 2107.0, 2106.0, 2101.0, 2031.0, 2031.0, 2031.0, 2031.0, 2031.0, 2029.0,
2029.0, 2029.0, 2029.0, 2029.0, 2028.0, 2028.0, 2028.0, 2028.0, 2027.0, 2026.0, 2026.0, 2026.0, 20
26.0, 2026.0, 2026.0, 2026.0, 2026.0, 2026.0, 2025.0, 2025.0, 2025.0, 2025.0, 2025.0, 2025.0, 202
5.0, 2025.0, 2025.0, 2025.0, 2025.0, 2025.0, 2025.0, 2024.0, 2024.0, 2024.0, 2024.0, 2024.0, 2024.
0, 2024.0]
```

In [23]:
```
# Calculate the 1% and 99% quantiles
lower_bound = df['orgyear'].quantile(0.01)
upper_bound = df['orgyear'].quantile(0.99)

# Clip the values
df['orgyear'] = df['orgyear'].clip(lower=lower_bound, upper=upper_bound)
```

The orgyear column contained extreme values (e.g., 0.0,1,0, 200.0) and (20165.0, 20165.0, 2204.0) that were not realistic representations of years. Such outliers could distort statistical analysis and model predictions.

By clipping values to the 1st and 99th percentiles, we ensure that the data remains within a reasonable range, reducing the impact of outliers without completely removing them.

In [24]:
```
df.isnull().sum()
```

Out[24]:
```
orgyear                  86
ctc                       0
job_position          52548
ctc_updated_year          0
email_hash_cleaned        0
company_hash_cleaned      0
dtype: int64
```

In [25]:
```
# Filter valid rows
valid_data = df[~df['job_position'].isnull() & ~df['company_hash_cleaned'].isnull()]

# Group by Company_hash and Job_position to calculate median of orgyear
median_values = valid_data.groupby(['company_hash_cleaned', 'job_position'])['orgyear'].agg('me

# Define a function to impute missing values
def impute_orgyear(row):
    if pd.isnull(row['orgyear']):
        key = (row['company_hash_cleaned'], row['job_position'])
        return median_values.get(key, np.nan)
    return row['orgyear']

df['orgyear'] = df.apply(impute_orgyear, axis=1)

print("Remaining null values in 'orgyear':", df['orgyear'].isnull().sum())
```

```
Remaining null values in 'orgyear': 55
```

In [26]:
```
# Calculate the median of 'orgyear' for each Company_hash
company_median = df.groupby('company_hash_cleaned')['orgyear'].median()

# Define a function to impute missing values with the company-based median
def impute_orgyear_with_company_median(row):
    if pd.isnull(row['orgyear']):
        return company_median.get(row['company_hash_cleaned'], np.nan)
    return row['orgyear']

df['orgyear'] = df.apply(impute_orgyear_with_company_median, axis=1)

print("Remaining null values in 'orgyear':", df['orgyear'].isnull().sum())
```

```
Remaining null values in 'orgyear': 26
```

In [27]:
```
overall_median = df['orgyear'].median()
df['orgyear'].fillna(overall_median, inplace=True)

print("Remaining null values in 'orgyear':", df['orgyear'].isnull().sum())
```

```
Remaining null values in 'orgyear': 0
```

In [28]:
```
df.isnull().sum()
```

Out[28]:
```
orgyear                 0
ctc                     0
job_position        52548
ctc_updated_year        0
email_hash_cleaned      0
company_hash_cleaned    0
dtype: int64
```

In [29]:
```
df.sample(10)
```

Out[29]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_ha
86519	2019.0	430000	FullStack Engineer	2021.0	7a853c39e2a810a5a9b22d309be3bea65828aac067c396...	yvjo mhoxztc
75949	2020.0	700000	NaN	2021.0	df45527b1fddb674888031d048f457b54e51ae586c9c46...	nvnv
19715	2016.0	440000	NaN	2020.0	d288b322086a92d5584c4a8046cf53ab3ae13e9f8ff281...	nvnv
19183	2020.0	1700000	FullStack Engineer	2019.0	fdf036b89af70a91270ccbace67966e80ae4ab6016c975...	
9232	2018.0	300000	FullStack Engineer	2021.0	cc77883269efc901944b90703528f68a84b3bad4e3b796...	
75161	2018.0	1300000	NaN	2021.0	632c5cf6490688be69f5d2ae99e2e3507882834b8e4aa5...	bvsx
86251	2013.0	2280000	NaN	2020.0	c1ed3565efe29f785dfdf7d52476315ca67de1eedacb9a...	
111917	2017.0	2800000	Backend Engineer	2020.0	a70f63c100a45b3fa09ad6fc02b8c0f7131369e5fee3e6...	
156200	2016.0	480000	iOS Engineer	2019.0	1dc10618109ad831e7617813e60879d06f31217e19bbc6...	uyvqbvvwt
17915	2018.0	500000	Frontend Engineer	2021.0	84752d34c5551b4ef15af8d6506b8c12fbc515657dad9a...	

In [30]:
```
1 df.dtypes
```

Out[30]:
```
orgyear                float64
ctc                      int64
job_position            object
ctc_updated_year       float64
email_hash_cleaned      object
company_hash_cleaned    object
dtype: object
```

In [31]:
```
1  # Group by 'company_hash_cleaned' and 'ctc' and calculate the mode of 'job_position'
2  job_position_mode = df.dropna(subset=['job_position']).groupby(['company_hash_cleaned', 'ctc'])
3
4  # Define a function to impute missing 'job_position'
5  def impute_job_position(row):
6      if pd.isnull(row['job_position']):
7          key = (row['company_hash_cleaned'], row['ctc'])
8          return job_position_mode.get(key, np.nan)
9      return row['job_position']
10
11 df['job_position'] = df.apply(impute_job_position, axis=1)
12
13 print("Remaining null values in 'job_position':", df['job_position'].isnull().sum())
```

```
Remaining null values in 'job_position': 12475
```

In [32]:
```
1 df['job_position'] = df['job_position'].fillna(df['job_position'].mode()[0])
2
3 # Check the result
4 print(df['job_position'].isnull().sum())
```

```
0
```

In [33]:
```
1 df.isnull().sum()
```

Out[33]:
```
orgyear                0
ctc                    0
job_position           0
ctc_updated_year       0
email_hash_cleaned     0
company_hash_cleaned   0
dtype: int64
```

In [34]:
```
1 df
```

Out[34]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_ha
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	at
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzv
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojz
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	
...	...	...	...	...	...	
205838	2008.0	220000	Backend Engineer	2019.0	70027b728c8ee901fe979533ed94ffda97be08fc23f33b...	
205839	2017.0	500000	FullStack Engineer	2020.0	7f7292ffad724ebbe9ca860f515245368d714c84705b42...	
205840	2021.0	700000	Backend Engineer	2021.0	cb25cc7304e9a24facda7f5567c7922ffc48e3d5d6018c...	
205841	2019.0	5100000	Backend Engineer	2019.0	fb46a1a2752f5f652ce634f6178d0578ef6995ee59f6c8...	zgn
205842	2014.0	1240000	Backend Engineer	2016.0	0bcfc1d05f2e8dc4147743a1313aa70a119b41b30d4a1f...	bg

205809 rows × 6 columns

➞ Making some new features like adding 'Years of Experience' column by subtracting orgyear from current year

In [35]:
```
1 from datetime import datetime
2
3 current_year = datetime.now().year
4
5 # Subtract 'orgyear' from the current year to calculate years of experience
6 df['Years_of_Experience'] = current_year - df['orgyear']
7
8 (df.head())
```

Out[35]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cl
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeq r
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpq
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen

➢ Manual Clustering on the basis of learner's company, job position and years of experience

➞ Getting the 5 point summary of CTC (mean, median, max, min, count etc) on the basis of Company, Job Position, Years of Experience

In [36]:
```
Group by 'company_hash_cleaned', 'job_position', and 'experience' to get the 5-point summary fo
ctc_summary = df.groupby(['company_hash_cleaned', 'job_position', 'Years_of_Experience'])['ctc'
```

➜ Merging the same with original dataset carefully and creating some flags showing learners with CTC greater than the Average of their Company's department having same Years of Experience - Call that flag designation with values [1,2,3]

In [37]:
```
# Calculate the average CTC for each combination of company and Years_of_Experience
avg_ctc = df.groupby(['company_hash_cleaned', 'Years_of_Experience'])['ctc'].mean().reset_index
avg_ctc.rename(columns={'ctc': 'avg_ctc'}, inplace=True)

# Merge the average CTC with the original dataset
df = pd.merge(df, avg_ctc, on=['company_hash_cleaned', 'Years_of_Experience'], how='left')

def assign_flag(row):
    if row['ctc'] > row['avg_ctc']:
        return 1
    elif row['ctc'] == row['avg_ctc']:
        return 2
    else:
        return 3

df['designation'] = df.apply(assign_flag, axis=1)

df.head()
```

Out[37]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeg rx
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpg
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen

➜ Doing above analysis at Company & Job Position level. Name that flag Class with values [1,2,3]

In [38]:

```
# Calculate the average CTC for each combination of company and job position and years of exper
avg_ctc_job_position = df.groupby(['company_hash_cleaned', 'job_position', 'Years_of_Experience
avg_ctc_job_position.rename(columns={'ctc': 'avg_ctc_job_position'}, inplace=True)

#  Merge the average CTC with the original dataset
df = pd.merge(df, avg_ctc_job_position, on=['company_hash_cleaned', 'job_position', 'Years_of_E

#  Create a flag (Class) based on whether learner's CTC is greater than, equal to, or less than
def assign_class_flag(row):
    if row['ctc'] > row['avg_ctc_job_position']:
        return 1
    elif row['ctc'] == row['avg_ctc_job_position']:
        return 2
    else:
        return 3

df['Class'] = df.apply(assign_class_flag, axis=1)

df.head()
```

Out[38]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeg rx
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpg
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen

In [39]:

```
# Convert avg_ctc and avg_ctc_job_position to integer format
df['avg_ctc'] = df['avg_ctc'].astype(int)
df['avg_ctc_job_position'] = df['avg_ctc_job_position'].astype(int)

df.head()
```

Out[39]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeg rx
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpg
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen

→ Repeating the same analysis at the Company level. Name that flag Tier with values [1,2,3]

In [40]:
```
# Calculate average CTC per company
avg_ctc_per_company = df.groupby('company_hash_cleaned')['ctc'].mean().reset_index()
avg_ctc_per_company.columns = ['company_hash_cleaned', 'avg_ctc_company']

# Merge the average CTC per company with the original dataframe
df = pd.merge(df, avg_ctc_per_company, on='company_hash_cleaned', how='left')

def assign_tier(row):
    if row['ctc'] > row['avg_ctc_company']:
        return 1
    elif row['ctc'] == row['avg_ctc_company']:
        return 2
    else:
        return 3

df['Tier'] = df.apply(assign_tier, axis=1)

df.head()
```

Out[40]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeg rx
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpg
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen

In [41]:
```
df['avg_ctc_company'] = df['avg_ctc_company'].astype(int)
```

In [42]:
```
df.head()
```

Out[42]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeg rx
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpg
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen

In [43]:
```
# Columns to retain
columns_to_keep = [
    'orgyear', 'ctc', 'job_position', 'ctc_updated_year',
    'email_hash_cleaned', 'company_hash_cleaned', 'Years_of_Experience', 'Class', 'Tier'
]

df2 = df[columns_to_keep]

df2.head()
```

Out[43]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeg rx
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpg
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen s

➢ Based on the manual clustering done so far, answering few questions like

➔ Top 10 employees (earning more than most of the employees in the company) - Tier 1

In [44]:
```
# Filter for Tier 1 employees
tier1_employees = df2[df2['Tier'] == 1]

# Sort by CTC in descending order
top_10_tier1_employees = tier1_employees.sort_values(by='ctc', ascending=False).head(10)

top_10_tier1_employees[['ctc','job_position','ctc_updated_year','email_hash_cleaned','company_h
```

Out[44]:

	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
117626	255555555	Backend Engineer	2016.0	5b4bed51797140db4ed52018a979db1e34cee49e27b488...	obvqnuqx
12646	200000000	Other	2020.0	0f0d1bf4233dadef653775c3f981f0ccde1dc20df36c43...	rtsvng ytvrny ntwyz
126131	200000000	Backend Engineer	2019.0	7683974378d0f5bacf95632f130a60cfa3ca39e368eec9...	
27075	200000000	Engineering Leadership	2020.0	0f0e7a9db34d1317498d9378a1bd0150bfa022ac6ba3a0...	q
17835	200000000	Backend Engineer	2019.0	a35a5abbe9fb056421bdd9aca4440acfb93e37c823564d...	
78783	200000000	Other	2020.0	76708a11cb61a030ff3da827b0fd19aff536c3793c1816...	
7345	200000000	Other	2020.0	68aa38470922a03f6022280b2a13c6f5ab6a717f70c77a...	
61281	200000000	Backend Engineer	2019.0	2f9a4241053f76b2f8c50ea593a90586d38b3f0e08c141...	vwwt
61288	200000000	Other	2019.0	3c85c094eaeb923add569ed91b8fbd6f8d8e8194dbaff5...	vb
2279	200000000	Other	2020.0	1f8e216b2328e7764f79dbd66deaf008b409870ae992fe...	grd s

➔ Top 10 employees of data science in each company earning more than their peers - Class 1

In [45]:

```
# Step 1: Define the list of data science-related job positions
data_science_roles = [
    "data scientist", "data analyst", "ml engineer", "intern-data analyst",
    "data science analyst", "data analystmachine learning engineer intern",
    "data scientist ii", "machine learning intern", "data scientist 2",
    "machine learning developer", "senior member of technical staff - r&d - machine learning",
    "machine learning developer", "ai engineer", "applied scientist"
]

data_science_employees = df2[df2['job_position'].str.lower().isin(data_science_roles)]

class1_data_science_employees = data_science_employees[data_science_employees['Class'] == 1]

class1_top_10_per_company = (
    class1_data_science_employees
    .sort_values(by=['company_hash_cleaned', 'ctc'], ascending=[True, False])
    .groupby('company_hash_cleaned')
    .head(10)
)

class1_top_10_per_company[['company_hash_cleaned','job_position','email_hash_cleaned','ctc','Cl
```

Out[45]:

	company_hash_cleaned	job_position	email_hash_cleaned	ctc	Class	Tier
142687	247vx	Data Scientist	9d2537610d57179230806bb77258f63c3134b8fde9aa3a...	2600000	1	1
164811	3p ntwyzgrgsxto	Data Scientist	af617ba27ec944771314f1c2d739b8208d2b3337800f8f...	1800000	1	1
28214	adw ntwyzgrgsj	Data Scientist	8a1e6a36db44c5ed8abe45ae0889577b2454e9025e7079...	2350000	1	1
59357	adw ntwyzgrgsj	Data Scientist	8a1e6a36db44c5ed8abe45ae0889577b2454e9025e7079...	2350000	1	1
36273	adw ntwyzgrgsj	Data Scientist	1f0a9f3b6a3de411b2ceece85ef7ef095278bf45496b0d...	1220000	1	3
...	...	...	...	...	...	...
99085	zxtrotz	Data Scientist	515862ad8c8c33263846231044741bfc177af2cddcf00f...	1000000	1	3
122850	zxtrotz	Data Scientist	515862ad8c8c33263846231044741bfc177af2cddcf00f...	1000000	1	3
28694	zxtrotz	Data Scientist	2182cd4f16b2a915d6c53f901f9911bb6b3b22caaaa0ae...	750000	1	3
111817	zxtrotz	Data Scientist	163e546c8418ddc4b300471c1472044f582cd3be008da1...	700000	1	3
109463	zxxn ntwyzgrgsxto rxbxnta	Data Scientist	06c1a29f43790b3b70b8dad94203cc09c30d6dab797b74...	1200000	1	3

984 rows × 6 columns

➜ Bottom 10 employees of data science in each company earning less than their peers - Class 3

In [46]:

```
#  Filter data science employees
data_science_employees = df2[df2['job_position'].str.lower().isin(data_science_roles)]

#  Filter for Class 3 employees
class3_data_science_employees = data_science_employees[data_science_employees['Class'] == 3]

class3_bottom_10_per_company = (
    class3_data_science_employees
    .sort_values(by=['company_hash_cleaned', 'ctc'], ascending=[True, True])
    .groupby('company_hash_cleaned')
    .head(10)
)

class3_bottom_10_per_company[['company_hash_cleaned','job_position','email_hash_cleaned','ctc',
```

Out[46]:

	company_hash_cleaned	job_position	email_hash_cleaned	ctc	Class	Tier
114743	247vx	Data Scientist	337d4da759d19f50c35655af7fedebfaef1d07842d418d...	2500000	3	1
106363	3p ntwyzgrgsxto	Data Scientist	583a9600d8e74d5f3f6627da6b4ff3c466bf5cfee5ae9f...	1200000	3	1
121786	adw ntwyzgrgsj	Data Scientist	6ca1d030df1927f2c0904548a99dea059fd0aeaf39cb21...	700000	3	3
80160	adw ntwyzgrgsj	Data Scientist	1443f6d836c4dc24a7a79f7f81702e6b684abdd22ea50e...	800000	3	3
160527	adw ntwyzgrgsj	Data Scientist	9d8dcc11f1524e2b9837295c74aa304420812e4044b2d5...	880000	3	3
...	...	...	...	...	...	...
168528	zxtrotz	Data Scientist	01717d934c1c75cacd31e29f8adcb5c109c627f7f26214...	650000	3	3
197876	zxtrotz	Data Scientist	af256544a43a5902d769859c21e05df919f05b490a227a...	650000	3	3
56132	zxtrotz	Data Scientist	2b7979bb62110fbb5e97f3f7f25d79f102536d3b84d538...	1090000	3	3
103131	zxxn ntwyzgrgsxto rxbxnta	Data Scientist	5ab93fd511bceaa6da5f855d160de306a04df9951f5978...	800000	3	3
141284	zxxn ntwyzgrgsxto rxbxnta	Data Scientist	5ab93fd511bceaa6da5f855d160de306a04df9951f5978...	800000	3	3

1033 rows × 6 columns

→ Bottom 10 employees (earning less than most of the employees in the company)- Tier 3

In [47]:
```
# Step 1: Filter for Tier 3 employees
tier3_employees = df2[df2['Tier'] == 3]

# Step 2: Sort by company and CTC in ascending order
tier3_sorted = tier3_employees.sort_values(by=['company_hash_cleaned', 'ctc'], ascending=[True,

tier3_bottom_10_per_company = tier3_sorted.groupby('company_hash_cleaned').head(10)

tier3_bottom_10_per_company[['company_hash_cleaned','job_position','email_hash_cleaned','ctc','
```

Out[47]:

	company_hash_cleaned	job_position	email_hash_cleaned	ctc	Class	Tier
74429	01 ojztqsj	Android Engineer	819789ff4068fd5c8facf8a5074cdd2e1ff989c95ae02c...	270000	2	3
2208	1	Backend Engineer	8cc7aba49e96a0a80f7ed6c2ed79bc1d1e81171a28445c...	100000	2	3
102576	1 jtvq	Backend Engineer	26a4487b5be40da1e942bf3bc1189172727de0e1c74c5e...	660000	3	3
99790	10nxbto	Frontend Engineer	951189c83134ac5394c85c66d0e96f2a0a242f3d99b87d...	400000	2	3
108925	10nxbto	FullStack Engineer	2dde60f4a510b412ac4e8011e806282ecb181a51081751...	400000	3	3
...	...	...	...	...	...	...
99376	zxztrtvuo	Other	31f3aea195401c4f20d0e910d2ac18d0f85d9664165afa...	450000	2	3
143298	zxztrtvuo	Other	f230cabdfbe43c7fe4ba66629b0fa9e058b1fd613dc232...	450000	2	3
155999	zxztrtvuo	Other	e2d27a8acde6484d35b69a75d9ecbc8b1f541223b3a296...	450000	2	3
150675	zyco xzaxv	Other	ed0a1202c31bdee343662f5517fc467fb8b96ccaf8e3eb...	500000	3	3
14670	zz	Backend Engineer	d6923a6f81c7b36615d9f14349fe01aec442029b2c502f...	500000	2	3

32199 rows × 6 columns

➜ Top 10 companies (based on their CTC)

In [48]:
```
# Step 1: Group by company and calculate the average CTC
avg_ctc_per_company = df2.groupby('company_hash_cleaned')['ctc'].mean().reset_index()

# Step 2: Sort the companies by average CTC in descending order
top_10_companies = avg_ctc_per_company.sort_values(by='ctc', ascending=False).head(10)

top_10_companies['ctc']=top_10_companies['ctc'].astype(int)

top_10_companies
```

Out[48]:

	company_hash_cleaned	ctc
30495	whmxw rgsxwo uqxcvnt rxbxnta	1000150000
1218	aveegaxr xzntqzvnxgzvr hzxctqoxnj	250000000
28859	vuytrxgz ogenfvqto ucn rna	200000000
10126	ltnvxqfvjo	200000000
12221	nhqvmxn vx vooxonvzno	200000000
12070	ngfvqao xzaxv	200000000
12033	neny	200000000
12020	nco rgsxonxwo otqcxwto rxbxnta	200000000
11481	mvpyntq nqvaxzs	200000000
29075	vwwg	200000000

➜ Top 2 positions in every company (based on their CTC)

In [49]:
```
# Step 1: Group by company and job position to calculate the average CTC
avg_ctc_per_position = df2.groupby(['company_hash_cleaned', 'job_position'])['ctc'].mean().rese

# Step 2: Sort by company and average CTC in descending order
sorted_positions = avg_ctc_per_position.sort_values(by=['company_hash_cleaned', 'ctc'], ascendi

top_2_positions = sorted_positions.groupby('company_hash_cleaned').head(2)

top_2_positions
```

Out[49]:

	company_hash_cleaned	job_position	ctc
0	0	Other	100000.0
1	0000	Other	300000.0
3	01 ojztqsj	Frontend Engineer	830000.0
2	01 ojztqsj	Android Engineer	270000.0
4	05mz exzytvrny uqxcvnt rxbxnta	Backend Engineer	1100000.0
...	...	...	...
63059	zz	Other	1370000.0
63058	zz	Backend Engineer	500000.0
63060	zzb ztdnstz vacxogqj ucn rna	FullStack Engineer	600000.0
63061	zzgato	Backend Engineer	130000.0
63062	zzzbzb	Other	720000.0

46185 rows × 3 columns

➢ Data processing for Unsupervised clustering - Label encoding/ One- hot encoding, Standardization of data

In [50]:
```
df2.sample(5)
```

Out[50]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_has
79945	2018.0	400000	Support Engineer	2021.0	8b3cb641e7629dff1fa33f9986752aa5cca41f565353df...	
12627	2013.0	800000	Other	2020.0	ed46215156656b88c3adc7af15d085f2facbaf63a2df66...	
142767	2015.0	1200000	Frontend Engineer	2017.0	00d41e298341de078f127f8b5c330e9e7a8ae2984ecb23...	e n
77165	2019.0	3350000	Backend Engineer	2020.0	ff70620a833b6de3896141775bedfe1ab6a5e9b1c5c231...	
2441	2014.0	1200000	FullStack Engineer	2021.0	30a345c34c5b033e0e2417bca7310000d84d3580fb7fb1...	

In [51]:
```
import datetime

current_year = datetime.datetime.now().year

df2['Years_since_CTC_update'] = current_year - df2['ctc_updated_year']

df.head()
```

```
C:\Users\Prajwal\AppData\Local\Temp\ipykernel_21184\1714231291.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)
  df2['Years_since_CTC_update'] = current_year - df2['ctc_updated_year']
```

Out[51]:

	orgyear	ctc	job_position	ctc_updated_year	email_hash_cleaned	company_hash_cle
0	2016.0	1100000	Other	2020.0	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05...	atrgxnnt
1	2018.0	449999	FullStack Engineer	2019.0	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10...	qtrxvzwt xzeg rx
2	2015.0	2000000	Backend Engineer	2020.0	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9...	ojzwnvwn
3	2017.0	700000	Backend Engineer	2019.0	effdede7a2e7c2af664c8a31d9346385016128d66bbc58...	ngpg
4	2017.0	1400000	FullStack Engineer	2019.0	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520...	qxen

In [52]:
```
df2[['orgyear', 'Years_of_Experience']].corr()
```

Out[52]:

	orgyear	Years_of_Experience
orgyear	1.0	-1.0
Years_of_Experience	-1.0	1.0

Years_of_Experience is derived as Current Year - orgyear.

Including both in clustering would be redundant, as they represent the same feature but in opposite directions.

In [53]:
```
df2 = df2.drop(columns=['orgyear'])
```

In [54]:
```
df2[['ctc_updated_year', 'Years_since_CTC_update']].corr()
```

Out[54]:

	ctc_updated_year	Years_since_CTC_update
ctc_updated_year	1.0	-1.0
Years_since_CTC_update	-1.0	1.0

ctc_updated_year and Years_since_CTC_update are essentially the same information in different forms, similar to the relationship between orgyear and Years_of_Experience.

In [55]:
```
df2 = df2.drop(columns=['ctc_updated_year'])
df2.drop(columns=['email_hash_cleaned'], inplace = True)

df2.head()
```

Out[55]:

	ctc	job_position	company_hash_cleaned	Years_of_Experience	Class	Tier	Years_since_CTC_update
0	1100000	Other	atrgxnnt xzaxv	9.0	2	3	5.0
1	449999	FullStack Engineer	qtrxvzwt xzegwgbb rxbxnta	7.0	3	3	6.0
2	2000000	Backend Engineer	ojzwnvwnxw vx	10.0	2	2	5.0
3	700000	Backend Engineer	ngpgutaxv	8.0	3	3	6.0
4	1400000	FullStack Engineer	qxen sqghu	8.0	2	1	6.0

In [56]:
```
col=['job_position', 'company_hash_cleaned', 'Class', 'Tier']
for i in col:
    print(i," ",df2[i].nunique())
```

```
job_position   1016
company_hash_cleaned   37300
Class   3
Tier   3
```

In [57]:
```
df2['Years_of_Experience'].min()
```

Out[57]: 4.0

In [58]:
```
df2['Years_of_Experience'].max()
```

Out[58]: 24.0

In [59]:
```
df2.shape
```

Out[59]: (205809, 7)

copy

In [60]:
```
df3=df2.copy()
```

In [61]:
```
df3.sample(5)
```

Out[61]:

	ctc	job_position	company_hash_cleaned	Years_of_Experience	Class	Tier	Years_since_CTC_update
103187	800000	FullStack Engineer	exatrxnj xzntqzvnxgzvr	7.0	1	3	4.0
118584	360000	Non Coder	om lvxz	9.0	2	2	4.0
72469	720000	Frontend Engineer	bhqvvx xzegqbvnxgz ntwyzgrgsj	9.0	2	1	5.0
1414	1400000	Other	myvmyv vngbxw qtotvqwy wtznqt	15.0	2	3	5.0
193119	200000	Engineering Leadership	xxn xob ayvzmva	10.0	2	3	6.0

performing target encoding on the job_position and company_hash_cleaned columns, replacing each with the average CTC for that respective job position and company:

In [62]:
```
# Target encode 'job_position' by replacing it with its average CTC
job_position_ctc = df3.groupby('job_position')['ctc'].mean().reset_index()
job_position_ctc.rename(columns={'ctc': 'avg_ctc_job_position'}, inplace=True)

df3 = pd.merge(df3, job_position_ctc, on='job_position', how='left')

company_ctc = df3.groupby('company_hash_cleaned')['ctc'].mean().reset_index()
company_ctc.rename(columns={'ctc': 'avg_ctc_company'}, inplace=True)

df3 = pd.merge(df3, company_ctc, on='company_hash_cleaned', how='left')

df3.head()
```

Out[62]:

	ctc	job_position	company_hash_cleaned	Years_of_Experience	Class	Tier	Years_since_CTC_update	avg_ctc_job_p
0	1100000	Other	atrgxnnt xzaxv	9.0	2	3	5.0	3.2566
1	449999	FullStack Engineer	qtrxvzwt xzegwgbb rxbxnta	7.0	3	3	6.0	1.7275
2	2000000	Backend Engineer	ojzwnvwnxw vx	10.0	2	2	5.0	2.1044
3	700000	Backend Engineer	ngpgutaxv	8.0	3	3	6.0	2.1044
4	1400000	FullStack Engineer	qxen sqghu	8.0	2	1	6.0	1.7275

Scaling only continuous numerical columns: These would include ctc, Years_of_Experience, avg_ctc, avg_ctc_job_position, avg_ctc_company.

not scaling Class and Tier because they are ordinal features.

In [63]:
```
from sklearn.preprocessing import MinMaxScaler

# Define columns to scale
columns_to_scale = ['ctc', 'Years_of_Experience', 'Years_since_CTC_update', 'avg_ctc_job_positi

scaler = MinMaxScaler()

df3[columns_to_scale] = scaler.fit_transform(df3[columns_to_scale])

df3.head()
```

Out[63]:

	ctc	job_position	company_hash_cleaned	Years_of_Experience	Class	Tier	Years_since_CTC_update	avg_ctc_job_p
0	0.00110	Other	atrgxnnt xzaxv	0.25	2	3	0.166667	0.
1	0.00045	FullStack Engineer	qtrxvzwt xzegwgbb rxbxnta	0.15	3	3	0.333333	0.
2	0.00200	Backend Engineer	ojzwnvwnxw vx	0.30	2	2	0.166667	0.
3	0.00070	Backend Engineer	ngpgutaxv	0.20	3	3	0.333333	0.
4	0.00140	FullStack Engineer	qxen sqghu	0.20	2	1	0.333333	0.

In [64]:
```
df3.drop(['job_position', 'company_hash_cleaned'], axis=1, inplace=True)
```

In [65]:
```
df3.head()
```

Out[65]:

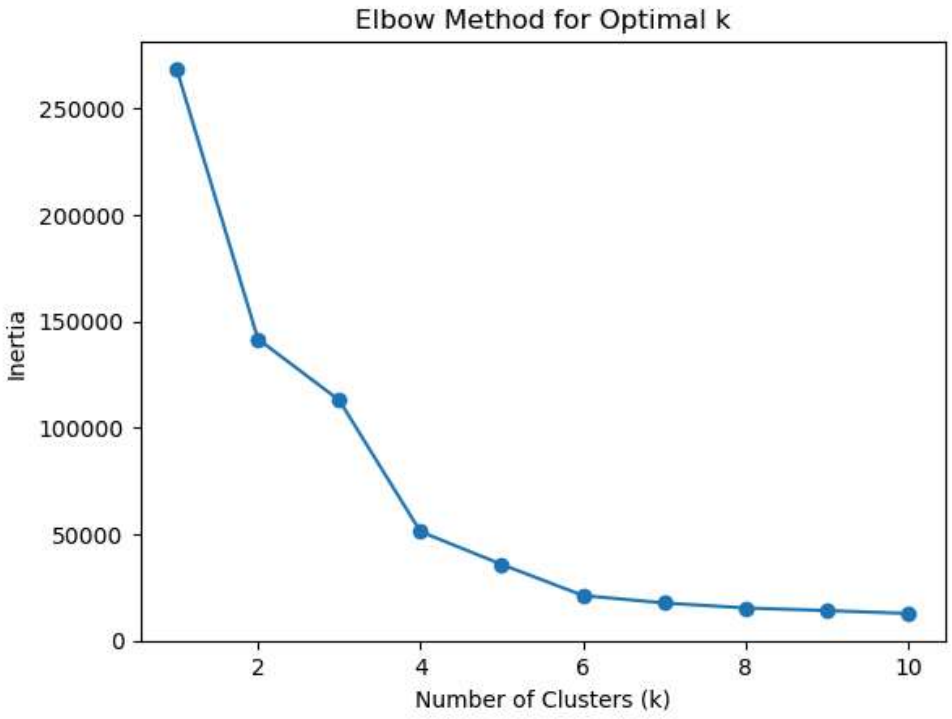
	ctc	Years_of_Experience	Class	Tier	Years_since_CTC_update	avg_ctc_job_position	avg_ctc_company
0	0.00110	0.25	2	3	0.166667	0.032547	0.001115
1	0.00045	0.15	3	3	0.333333	0.017255	0.002193
2	0.00200	0.30	2	2	0.166667	0.021025	0.002000
3	0.00070	0.20	3	3	0.333333	0.021025	0.001714
4	0.00140	0.20	2	1	0.333333	0.017255	0.000940

➢ Unsupervised Learning - Clustering

➞ Elbow method

To find the minimal number of clusters k for KMeans clustering, we can use the Elbow Method or Silhouette Score.

In [66]:
```
from sklearn.cluster import KMeans

# Calculate inertia for different values of k
inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(df3[['ctc', 'Years_of_Experience', 'Class', 'Tier', 'Years_since_CTC_update',
                    'avg_ctc_job_position', 'avg_ctc_company']])
    inertia.append(kmeans.inertia_)

# Plot the Elbow curve
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.show()
```



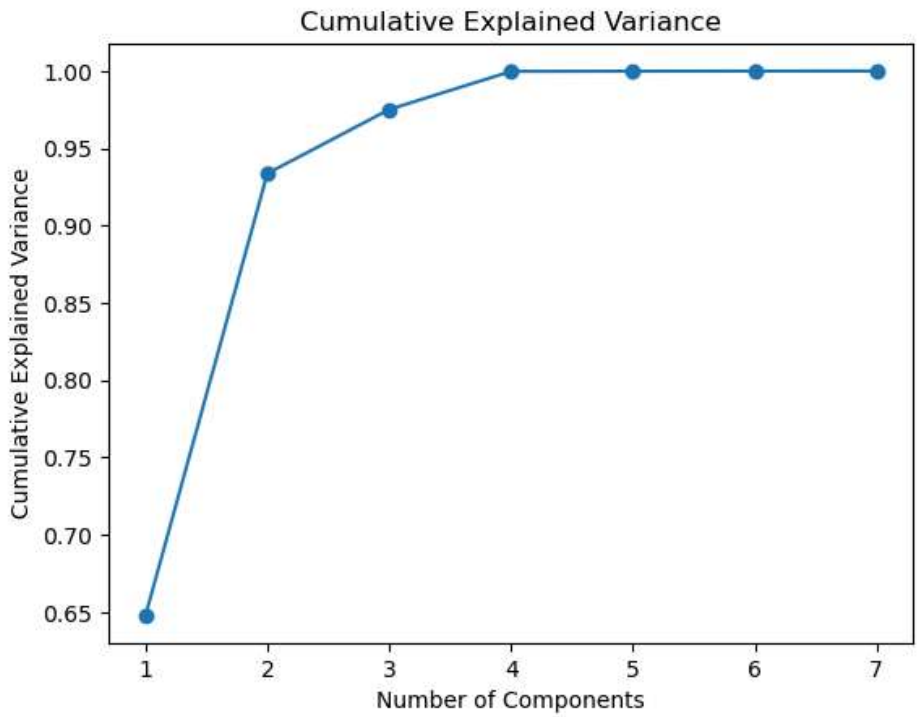
The elbow point appears to be around k=4 or k=5. This indicates that using 4 or 5 clusters likely provides a good balance

The Silhouette Score measures how similar each sample is to its own cluster compared to other clusters. The score ranges from -1 to 1

In [67]:

```
from sklearn.decomposition import PCA

pca = PCA()
pca.fit(df3[['ctc', 'Years_of_Experience', 'Class', 'Tier',
                                        'Years_since_CTC_update', 'avg_ctc_job_position',
                                        'avg_ctc_company']])

explained_variance = pca.explained_variance_ratio_

print("Explained Variance Ratio:", explained_variance)

import matplotlib.pyplot as plt
cumulative_variance = explained_variance.cumsum()
plt.plot(range(1, len(cumulative_variance)+1), cumulative_variance, marker='o')
plt.title("Cumulative Explained Variance")
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance")
plt.show()
```

```
Explained Variance Ratio: [6.47807186e-01 2.85942630e-01 4.13939985e-02 2.46267093e-02
 1.23745500e-04 8.52180163e-05 2.05130341e-05]
```



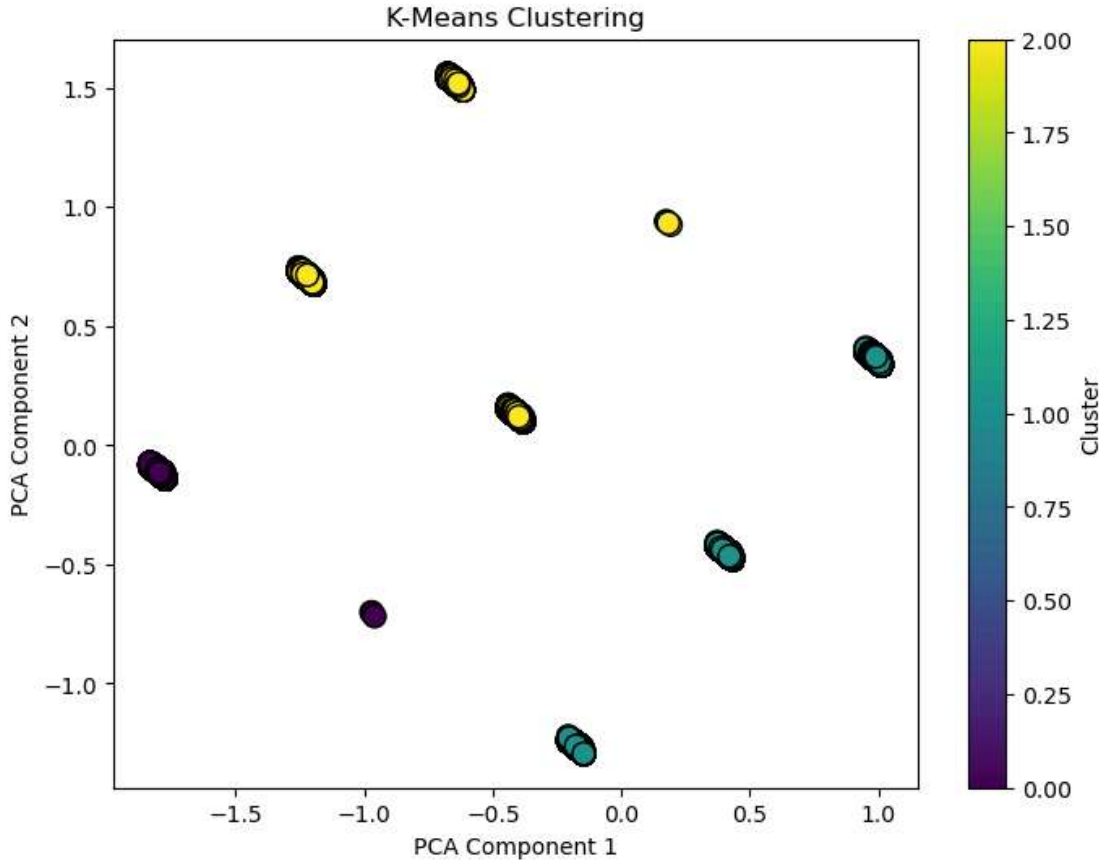
In [106]:

```
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

pca = PCA(n_components=3)
reduced_data = pca.fit_transform(df3[['ctc', 'Years_of_Experience', 'Class', 'Tier',
                                      'Years_since_CTC_update', 'avg_ctc_job_position',
                                      'avg_ctc_company']])

kmeans = KMeans(n_clusters=5, random_state=42)
cluster_labels = kmeans.fit_predict(reduced_data)

silhouette_avg = silhouette_score(reduced_data, cluster_labels)
print(f"Silhouette Score: {silhouette_avg}")
```

```
Silhouette Score: 0.6793786948323985
```

In [69]:

```
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
df3['Cluster'] = kmeans.fit_predict(df3[['ctc', 'Years_of_Experience', 'Class', 'Tier',
                                         'Years_since_CTC_update', 'avg_ctc_job_position',
                                         'avg_ctc_company']])

pca = PCA(n_components=3)
pca_components = pca.fit_transform(df3[['ctc', 'Years_of_Experience', 'Class', 'Tier',
                                         'Years_since_CTC_update', 'avg_ctc_job_position',
                                         'avg_ctc_company']])

# Plot the clusters in 2D
plt.figure(figsize=(8,6))
plt.scatter(pca_components[:, 0], pca_components[:, 1], c=df3['Cluster'], cmap='viridis', marke
plt.title("K-Means Clustering")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.colorbar(label="Cluster")
plt.show()
```



1. Interpret Clusters:

In [70]:
```
# Get the mean of features for each cluster
cluster_summary = df3.groupby('Cluster')[['ctc', 'Years_of_Experience', 'Class', 'Tier',
                                          'Years_since_CTC_update', 'avg_ctc_job_position',
                                          'avg_ctc_company']].mean()

cluster_summary
```

Out[70]:

Cluster	ctc	Years_of_Experience	Class	Tier	Years_since_CTC_update	avg_ctc_job_position	avg_ctc_compar
0	0.007628	0.326707	1.000000	1.000161	0.213550	0.022046	0.00201
1	0.000943	0.248239	2.332579	3.000000	0.225654	0.022431	0.00227
2	0.003411	0.375835	2.068587	1.550441	0.239528	0.023465	0.00234

2. Examine Cluster Centers:

In [71]:
```
# Get the cluster centers
cluster_centers = kmeans.cluster_centers_
print("Cluster Centers:")
cluster_centers
```

```
Cluster Centers:
```

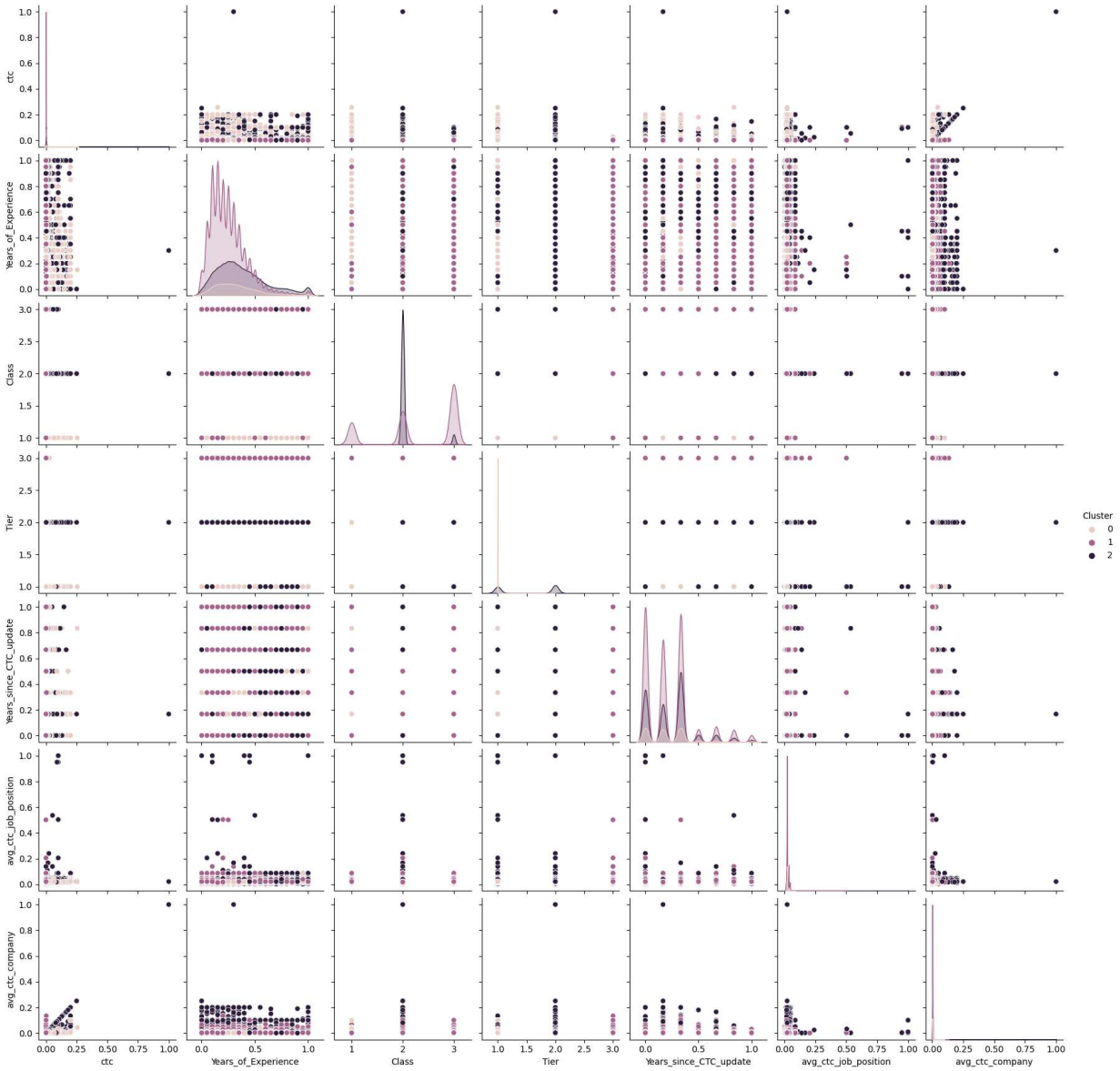
Out[71]:
```
array([[7.62789262e-03, 3.26706526e-01, 1.00000000e+00, 1.00016087e+00,
        2.13550324e-01, 2.20460188e-02, 2.01713547e-03],
       [9.42907317e-04, 2.48239367e-01, 2.33257929e+00, 3.00000000e+00,
        2.25653940e-01, 2.24310194e-02, 2.27235588e-03],
       [3.41059675e-03, 3.75834743e-01, 2.06858745e+00, 1.55044122e+00,
        2.39528043e-01, 2.34651961e-02, 2.34840262e-03]])
```

3. Visualize the Cluster Characteristics:

In [72]:

```
import seaborn as sns
sns.pairplot(df3, hue='Cluster', vars=['ctc', 'Years_of_Experience', 'Class', 'Tier',
                                       'Years_since_CTC_update', 'avg_ctc_job_position',
                                       'avg_ctc_company'])
plt.show()
```

```
D:\Anaconda\Lib\site-packages\seaborn\axisgrid.py:118: UserWarning: The figure layout has changed
to tight
  self._figure.tight_layout(*args, **kwargs)
```



General Trends:

‣ Years of Experience: There is a general positive correlation between years of experience and average CTC (avg_ctc_company, avg_ctc_job_position). This is expected as people with more experience tend to earn higher salaries.

‣ CTC Update: The distribution of "Years Since CTC Update" shows that many employees haven't had their salaries updated recently. This could be an area for HR to investigate.

‣ Class: There appears to be some variation in average CTC across different classes, though the differences are not very pronounced. Specific Observations:

‣ Tier: There is a clear trend of higher average CTC in Tier 1 companies compared to Tier 2 and Tier 3 companies.

‣ Class: Class 1 employees generally have higher average CTC compared to Class 2 and Class 3 employees.

- CTC Update: Employees with more recent CTC updates tend to have higher average CTC.

- Job Position: The average CTC varies significantly across different job positions, with some positions commanding much higher salaries than others.

4. Target Business Outcomes:

In [74]:
```
# For example, targeting cluster 0 for analysis
cluster_0 = df3[df3['Cluster'] == 0]
print(cluster_0[['ctc', 'Years_of_Experience', 'Class', 'Tier']].mean())  # Get mean features f
```

```
ctc                    0.007628
Years_of_Experience    0.326707
Class                  1.000000
Tier                   1.000161
dtype: float64
```

In [75]:
```
# For example, targeting cluster 1 for analysis
cluster_0 = df3[df3['Cluster'] == 1]
print(cluster_0[['ctc', 'Years_of_Experience', 'Class', 'Tier']].mean())  # Get mean features f
```

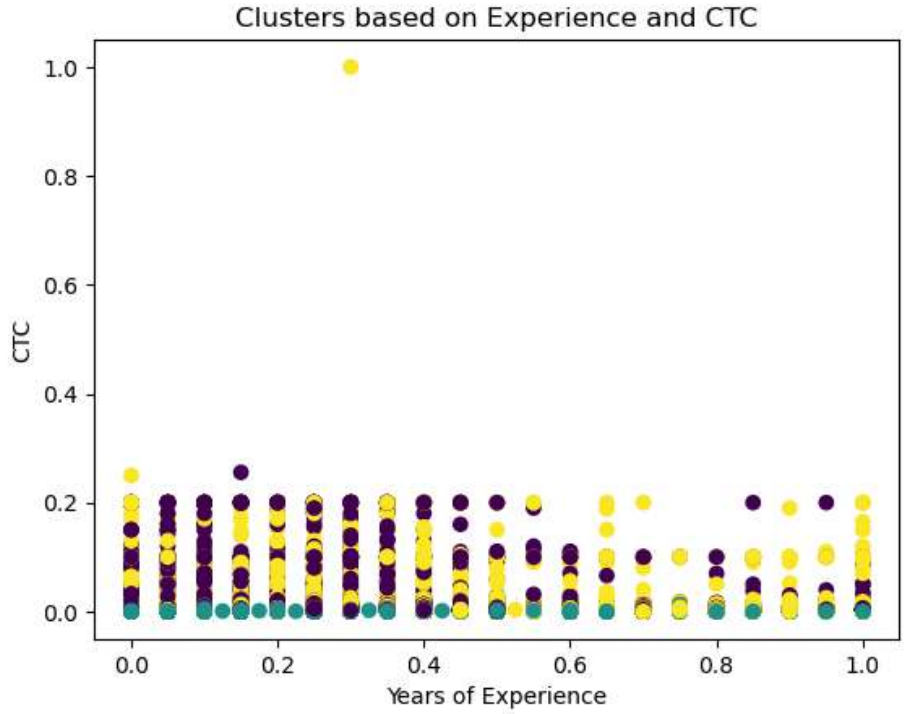
```
ctc                    0.000943
Years_of_Experience    0.248239
Class                  2.332579
Tier                   3.000000
dtype: float64
```

In [76]:
```
# For example, targeting cluster 2 for analysis
cluster_0 = df3[df3['Cluster'] == 2]
print(cluster_0[['ctc', 'Years_of_Experience', 'Class', 'Tier']].mean())  # Get mean features f
```

```
ctc                    0.003411
Years_of_Experience    0.375835
Class                  2.068587
Tier                   1.550441
dtype: float64
```

In [77]:

```
# Fit the KMeans model
kmeans = KMeans(n_clusters=3, random_state=42)
df3['Cluster'] = kmeans.fit_predict(df3[['ctc', 'Years_of_Experience', 'Class', 'Tier',
                                         'Years_since_CTC_update', 'avg_ctc_job_position',
                                         'avg_ctc_company']])

# Check the resulting clusters
df3[['ctc', 'Years_of_Experience', 'Cluster']].head()

# Plotting clusters (for example: Years_of_Experience vs ctc)
plt.scatter(df3['Years_of_Experience'], df3['ctc'], c=df3['Cluster'], cmap='viridis')
plt.title('Clusters based on Experience and CTC')
plt.xlabel('Years of Experience')
plt.ylabel('CTC')
plt.show()
```



Key Observations:

‣ Three Distinct Clusters: The data points are colored based on their assigned cluster (likely 3 clusters). This suggests that employees are grouped based on their experience and CTC levels.

‣ Cluster Distribution: The clusters appear to be somewhat overlapping, indicating that there's some variation within each cluster.

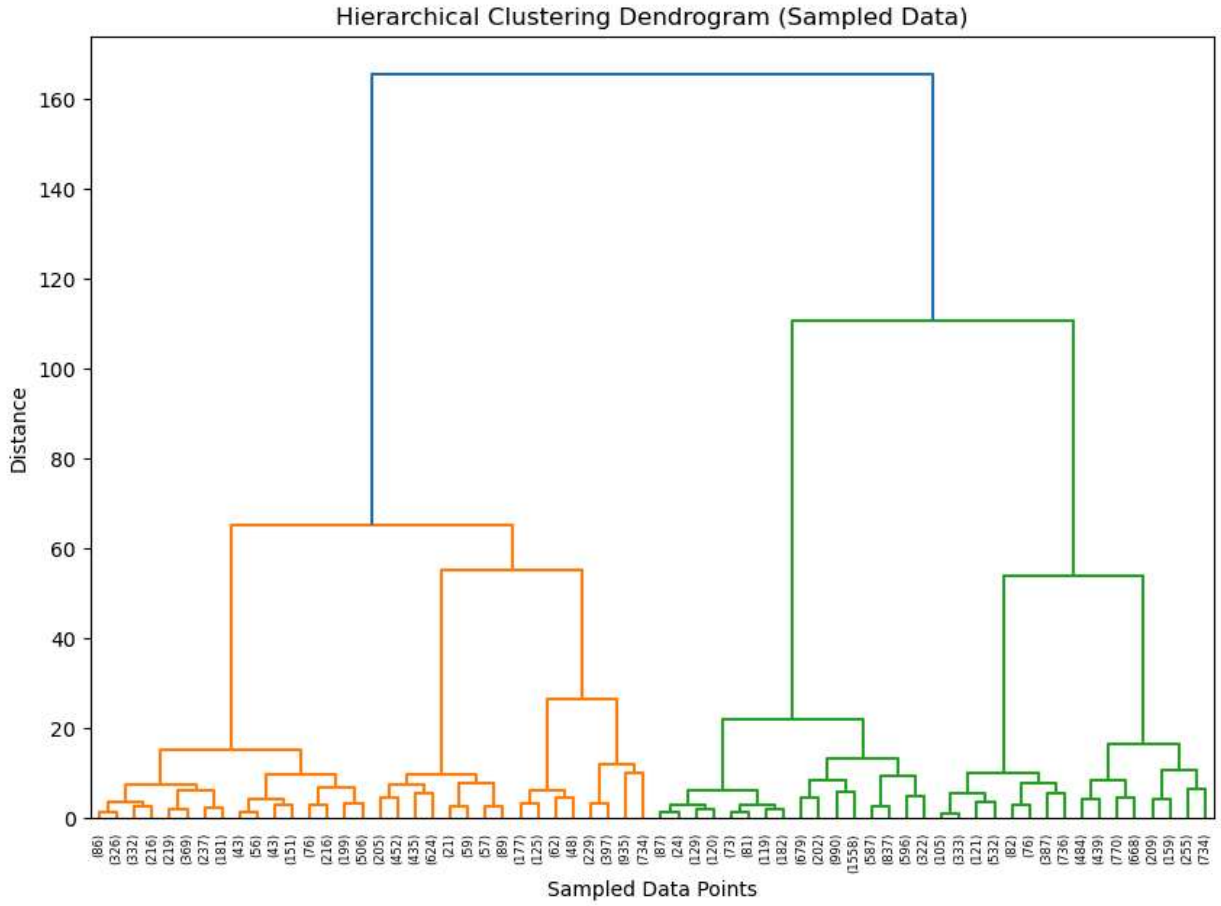
‣ Experience and CTC Relationship: There might be a general trend of higher CTC values as experience increases, although it's not a perfectly linear relationship.

Hierarchical clustering

is another powerful clustering method, particularly useful when the number of clusters (k) is not known in advance. It creates a hierarchy of clusters by either merging smaller clusters (agglomerative) or splitting larger ones (divisive).

In [78]:

```
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt

# Step 1: Random Sampling
sample_df = df3[['ctc', 'Years_of_Experience', 'Class', 'Tier',
                 'Years_since_CTC_update', 'avg_ctc_job_position',
                 'avg_ctc_company']].sample(frac=0.1, random_state=42)

# Step 2: Linkage Matrix Computation
# Using 'ward' linkage for variance minimization
linkage_matrix = linkage(sample_df, method='ward')

# Step 3: Plot Dendrogram for Visualizing Clusters
plt.figure(figsize=(10, 7))
dendrogram(linkage_matrix, truncate_mode='level', p=5)
plt.title('Hierarchical Clustering Dendrogram (Sampled Data)')
plt.xlabel('Sampled Data Points')
plt.ylabel('Distance')
plt.show()
```



Key Observations:

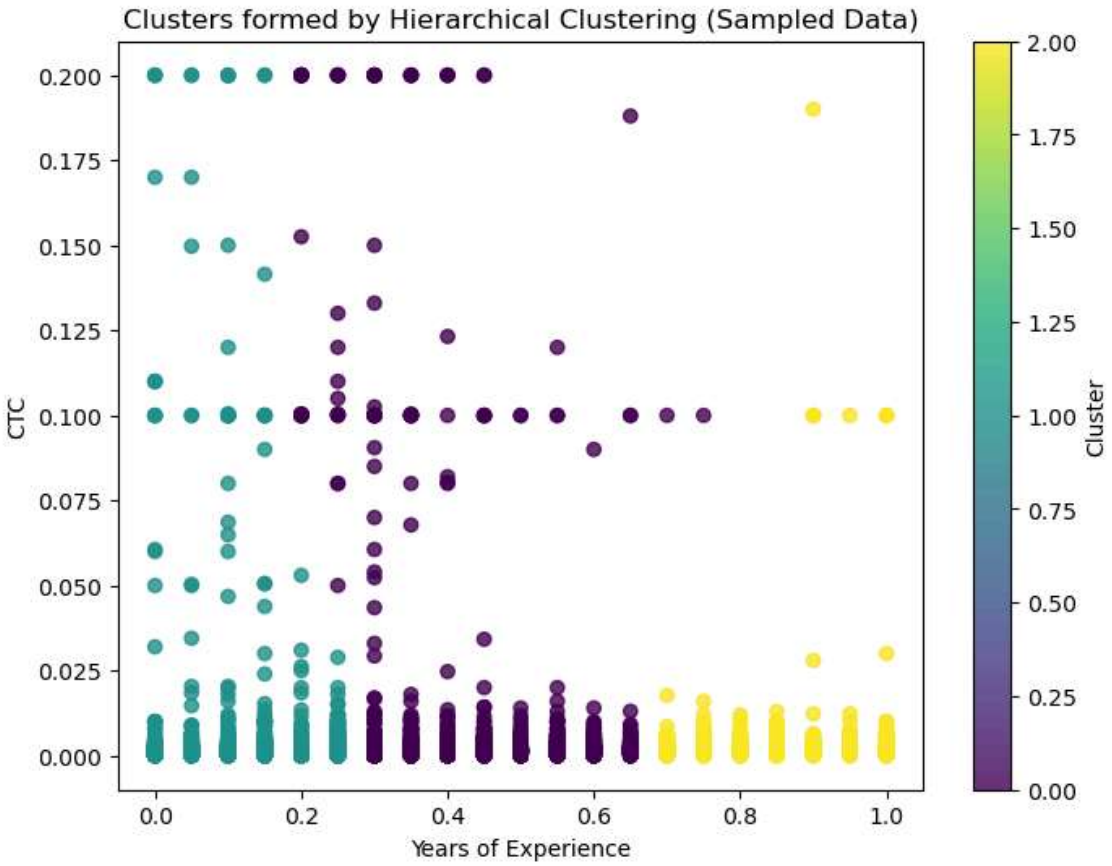
‣ Number of Clusters: The dendrogram suggests a natural division into three main clusters. This is evident from the three major vertical lines that connect the clusters.

‣ Cluster Sizes: The clusters appear to be of varying sizes. One cluster is significantly larger than the other two, indicating a potential imbalance in the data distribution.

‣ Distance Between Clusters: The vertical lines represent the distance between clusters. Longer lines indicate greater dissimilarity between clusters. The distance between the two smaller clusters is relatively small, suggesting they might be more closely related.

- Cluster Structure: The dendrogram shows a hierarchical structure, with smaller clusters merging to form larger ones. This reflects the process of agglomerative clustering, where individual data points are gradually grouped into larger clusters based on their similarity.

In [79]:
```
# Step 4: Agglomerative Clustering
from sklearn.cluster import AgglomerativeClustering

n_clusters = 3
hierarchical_clustering = AgglomerativeClustering(n_clusters=n_clusters, metric='euclidean', li
cluster_labels = hierarchical_clustering.fit_predict(sample_df[['Years_of_Experience', 'ctc']])

# Step 5: Add Clusters to Dataframe
sample_df['Cluster'] = cluster_labels

plt.figure(figsize=(8, 6))
plt.scatter(sample_df['Years_of_Experience'], sample_df['ctc'], c=cluster_labels, cmap='viridis
plt.title('Clusters formed by Hierarchical Clustering (Sampled Data)')
plt.xlabel('Years of Experience')
plt.ylabel('CTC')
plt.colorbar(label='Cluster')
plt.show()
```



Cluster Interpretations:

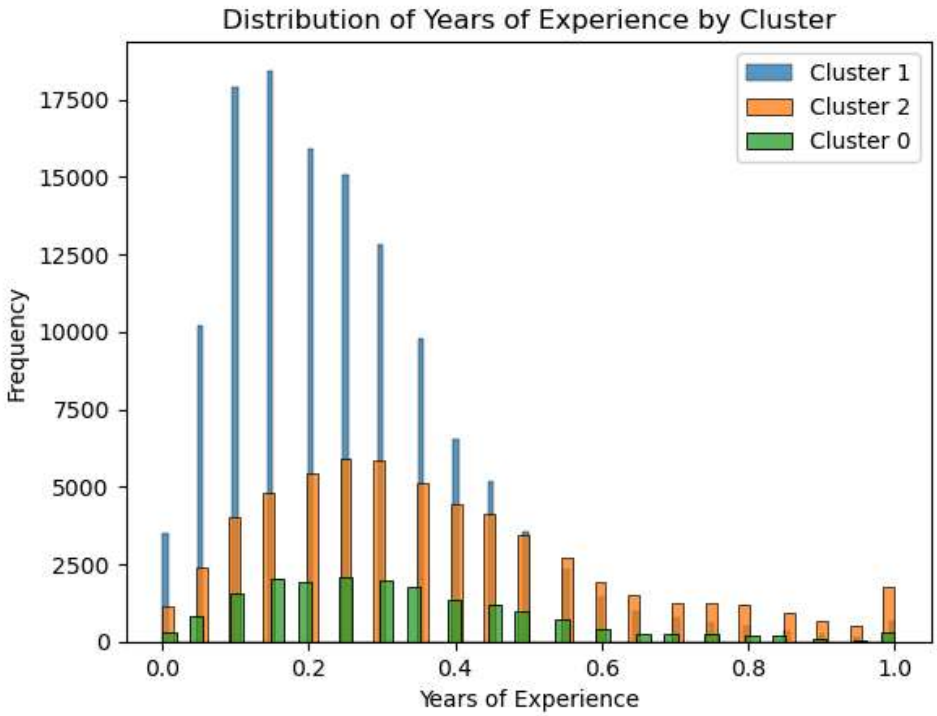
- Cluster 1 (e.g., Yellow): This cluster is likely to represent individuals with higher CTC and relatively higher years of experience. They might be senior professionals, individuals in high-paying roles, or those with strong career progression.

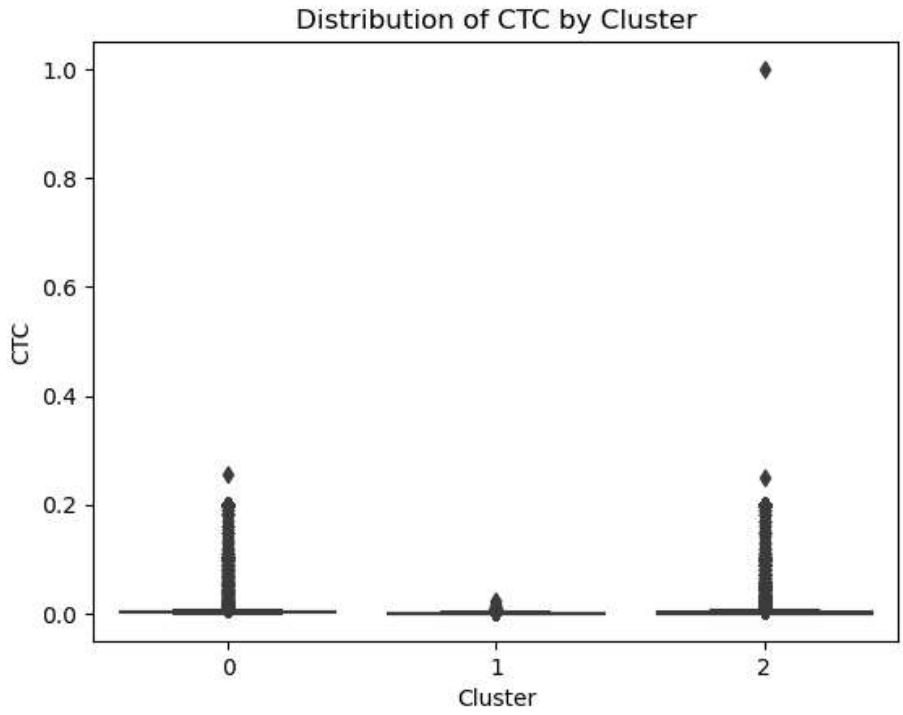
- Cluster 2 (e.g., Teal): This cluster might represent individuals with lower CTC and lower years of experience. This could include entry-level employees, those in roles with lower compensation, or individuals who are relatively new to the workforce.

- Cluster 3 (e.g., Purple/Dark Green): This cluster could represent individuals with moderate CTC and moderate years of experience. This might be a diverse group with varying levels of seniority and compensation.

In [80]:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Example: Histogram for 'Years_of_Experience' in each cluster
for cluster in df3['Cluster'].unique():
    sns.histplot(df3[df3['Cluster'] == cluster]['Years_of_Experience'], label=f'Cluster {cluste
plt.legend()
plt.xlabel('Years of Experience')
plt.ylabel('Frequency')
plt.title('Distribution of Years of Experience by Cluster')
plt.show()

# Example: Box plot for 'ctc' in each cluster
sns.boxplot(x='Cluster', y='ctc', data=df3)
plt.xlabel('Cluster')
plt.ylabel('CTC')
plt.title('Distribution of CTC by Cluster')
plt.show()
```





Key Observations:

‣ Cluster 1: This cluster appears to have the highest frequency of individuals with low to moderate years of experience. The distribution is skewed towards the left, indicating a higher concentration of individuals with fewer years of experience.

‣ Cluster 2: This cluster also shows a concentration of individuals with low to moderate years of experience, but to a lesser extent compared to Cluster 1. The distribution is also skewed towards the left.

‣ Cluster 0: This cluster has a more balanced distribution across different experience levels. There is a higher proportion of individuals with moderate to high years of experience compared to the other two clusters.

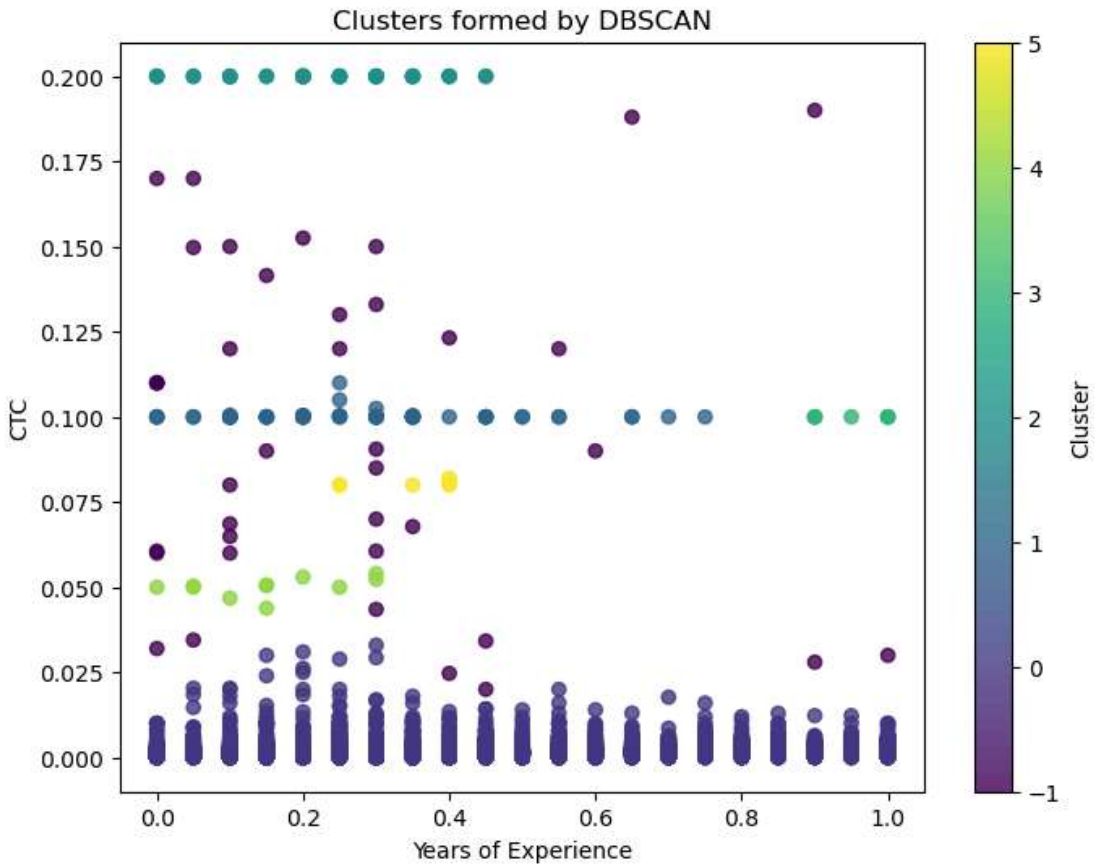
DBSCAN

is a density-based clustering method that groups points close to each other in dense regions and marks points in sparse regions as noise or outliers. It does not require specifying the number of clusters in advance, making it different from k-means or hierarchical clustering.

In [82]:
```
1 df3.shape
```

Out[82]: (205809, 8)

In [85]:
```
1
```




```
Number of noise points: 39
```

Relationship between Years of Experience and CTC:

‣ Positive Correlation: There appears to be a general positive correlation between "Years of Experience" and "CTC." As the "Years of Experience" increase, there's a tendency for "CTC" to also increase.

‣ Cluster-Specific Trends: The relationship between "Years of Experience" and "CTC" varies across clusters. Some clusters might show a stronger correlation than others.

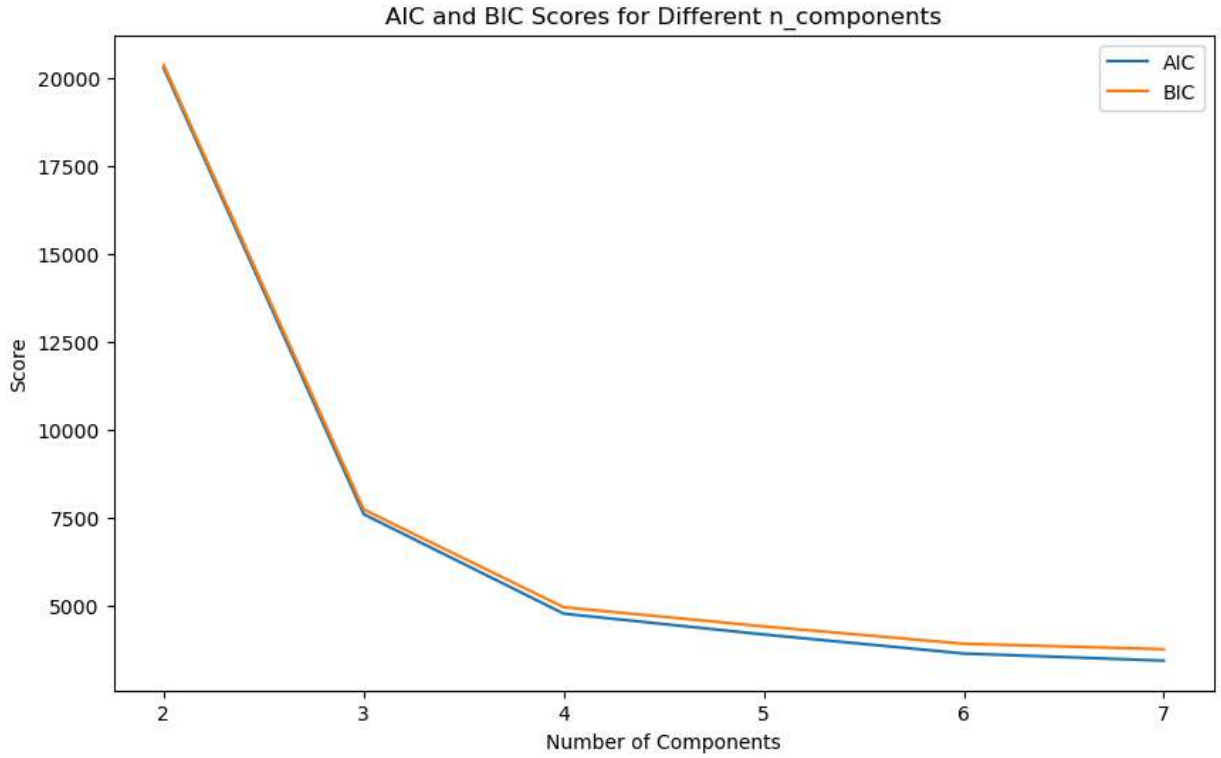
Cluster Characteristics:

‣ Dense Clusters: Some clusters appear to be more densely populated than others, indicating higher concentrations of data points in those regions.

‣ Cluster Shapes: The clusters exhibit various shapes, ranging from elongated to more circular. This suggests that the data distribution is not uniform and that DBSCAN has effectively captured the underlying density variations.

In [98]:

```
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt

aic_scores = []
bic_scores = []

components_range = range(2, 8)

for n in components_range:
    gmm = GaussianMixture(n_components=n, random_state=42)
    gmm.fit(scaled_data)
    aic_scores.append(gmm.aic(scaled_data))
    bic_scores.append(gmm.bic(scaled_data))

# Plot AIC and BIC to identify optimal n_components
plt.figure(figsize=(10, 6))
plt.plot(components_range, aic_scores, label='AIC')
plt.plot(components_range, bic_scores, label='BIC')
plt.xlabel('Number of Components')
plt.ylabel('Score')
plt.legend()
plt.title('AIC and BIC Scores for Different n_components')
plt.show()

```

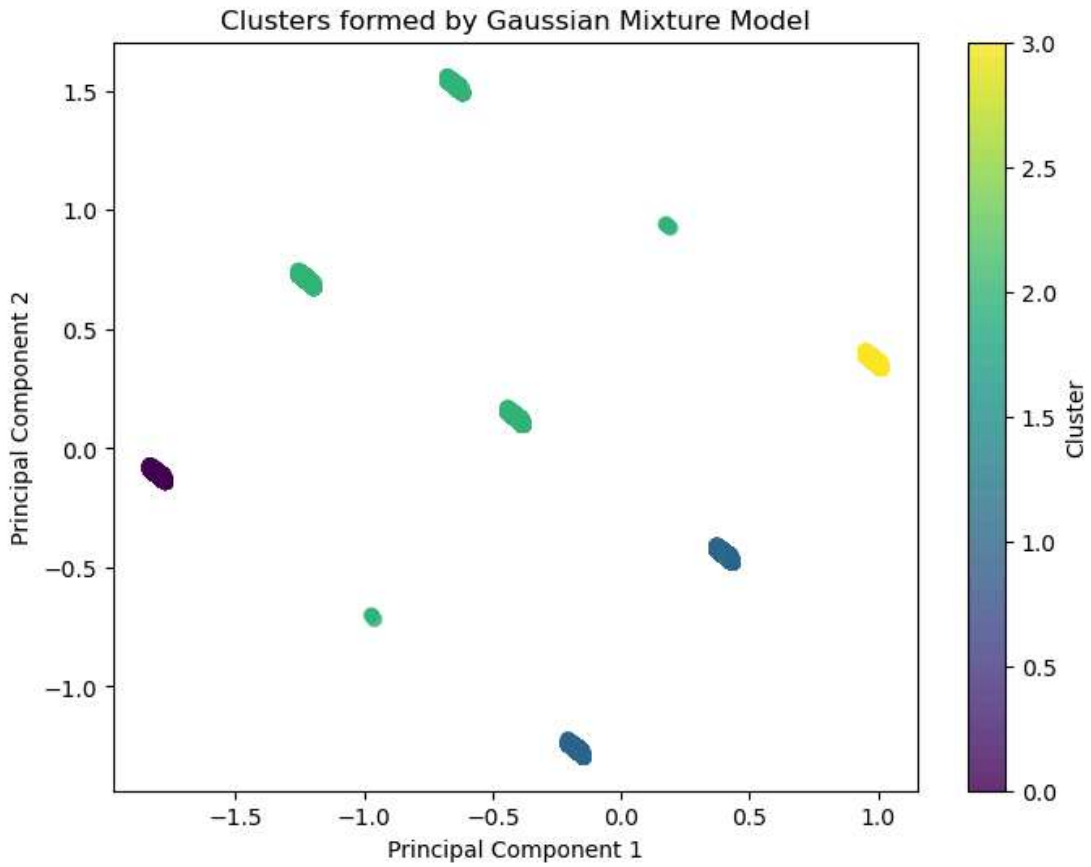


Interpretation:

‣ Model Selection: Based on the plot, the BIC score seems to suggest that the optimal number of components for the Gaussian Mixture Model (GMM) might be around 3 or 4. This is where the BIC score reaches its minimum value and then starts to increase.

In [99]:

```
from sklearn.mixture import GaussianMixture
from sklearn.decomposition import PCA

features = df3[['ctc', 'Years_of_Experience', 'Class', 'Tier',
                'Years_since_CTC_update', 'avg_ctc_job_position',
                'avg_ctc_company']]

pca = PCA(n_components=2)
reduced_data = pca.fit_transform(features)

n_components = 4
gmm = GaussianMixture(n_components=n_components, random_state=42)
cluster_labels = gmm.fit_predict(features)


df3['Cluster'] = cluster_labels

# Visualize Clusters
plt.figure(figsize=(8, 6))
plt.scatter(reduced_data[:, 0], reduced_data[:, 1], c=cluster_labels, cmap='viridis', alpha=0.8
plt.title('Clusters formed by Gaussian Mixture Model')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar(label='Cluster')
plt.show()

# Step 6: Evaluate Model Using BIC/AIC
print("BIC:", gmm.bic(features))
print("AIC:", gmm.aic(features))
```




```
BIC: -7336923.672992907
AIC: -7338387.235641056
```

Lower values of BIC and AIC generally indicate a better fit. In this case, both BIC and AIC are very large negative values, suggesting a good fit of the GMM to the data.

In [ ]:

```
1
```